POST AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

Faculty of Information Technology

INTELLIGENT SYSTEM DEVELOPMENT (INTEL_SYS_DEV)

Course Project Report: Assignment 04

Comparative Empirical Analysis of Deep Learning Framework Abstractions: From Vectorized NumPy to Keras and PyTorch

Investigating Architectural Invariance, Execution Latency, and Algorithmic Visibility across Tabular MLP and Visual CNN Paradigms

---

Student Name: Bùi Nguyên Hoàng Việt

Student ID: B23DCDT285

Class: E23CNPM02

Team: Individual Project

Lecturer: Dinh Que Tran, Ph.D., Assoc. Prof.

Semester: I.2026

Repository: https://github.com/minorin2847/INTEL_SYS_DEV_2026

Project Scope: A controlled comparative evaluation across three intelligent systems: metabolic risk screening (tabular MLP), apparel categorization (Fashion-MNIST grayscale CNN), and natural scene object recognition (CIFAR-10 color CNN). Each system is constructed across three independent software abstraction tiers to evaluate architectural invariance, execution efficiency, and diagnostic transparency.

Contents

1 Executive Summary and Core Empirical Findings **3**

2 Theoretical Foundations: Architectural Invariance vs. Abstraction Layers **4**
2.1 The Core Inquiry: Invariance across Software Abstractions 4
2.2 Conceptual Equivalence across Diverse Framework APIs 4
2.3 The Algorithmic Visibility and Developer Ergonomics Spectrum 5
2.4 Experimental Control and Fairness Protocol 5

3 Application I: Tabular Diabetes Metabolic Risk Screening (MLP) **6**
3.1 Problem Formulation and Clinical Scope 6
3.2 Data Acquisition and Longitudinal Schema Assembly 6
3.3 Exploratory Analysis, Sentinel Filtering, and Preprocessing 7
3.4 Feature Vector Encoding and Imbalance Mitigation 9
3.5 Unified Multi-Layer Perceptron Topology 9
3.6 Implementation Across Software Paradigms 10
3.6.1 Vectorized NumPy Implementation from First Principles 10
3.6.2 Declarative TensorFlow/Keras Implementation 10
3.6.3 Modular PyTorch Implementation 10
3.7 Structural and Operational Framework Mapping 11
3.8 Empirical Optimization Dynamics and Evaluation 11
3.9 Comparative Analysis and Architectural Trade-Offs 14

4 Application II: Fashion-MNIST Grayscale Apparel Categorization (CNN) **15**
4.1 Problem Formulation and Spatial Inductive Bias 15
4.2 Dataset Characterization and Partition Uniformity 15
4.3 Tensor Normalization and Memory Layout Alignment 16
4.4 Unified Two-Stage Convolutional Topology 17
4.5 Implementation Across Software Paradigms 17
4.5.1 Vectorized im2col Convolutional Engine in NumPy 17
4.5.2 Declarative TensorFlow/Keras Implementation 17
4.5.3 Modular PyTorch Implementation 17
4.6 Structural and Operational Framework Mapping 18
4.7 Empirical Optimization Dynamics and Evaluation 19
4.8 Comparative Analysis and Computational Scaling 22

5 Application III: CIFAR-10 Photographic Object Recognition (CNN) **23**
5.1 Problem Formulation and Multi-Channel Visual Complexity 23
5.2 Dataset Characterization and Chromatic Diversity 23
5.3 Multi-Channel Normalization and Layout Alignment 25
5.4 Unified Three-Channel Convolutional Topology 25
5.5 Implementation Across Software Paradigms 25
5.5.1 Multi-Channel im2col Convolutional Engine in NumPy 25
5.5.2 Declarative TensorFlow/Keras Implementation 25
5.5.3 Modular PyTorch Implementation 25
5.6 Structural and Operational Framework Mapping 26
5.7 Empirical Optimization Dynamics and Evaluation 27

5.8 Comparative Analysis and Workload-Driven Latency 30

6 Cross-Application Synthesis and Empirical Trends 31

7 Synthesis Discussion: Lessons Across Abstraction Levels 31
7.1 Validation of the Architectural Invariance Principle 31
7.2 Divergence in Computational Latency and Kernel Scaling 32
7.3 Hidden Mechanics Surfaced Through Manual Implementation 32

8 Conclusion and Key Takeaways 32

9 System Environment and Reproducibility Protocol 33
9.1 Software Environment and Deterministic Controls 33
9.2 Headless Reproduction and Artifact Verification 33

1 Executive Summary and Core Empirical Findings

This report investigates three distinct intelligent systems: screening for diabetes metabolic risk (tabular data, binary classification), Fashion-MNIST apparel categorization (single-channel grayscale imagery, 10-class classification), and CIFAR-10 photographic object recognition (three-channel RGB natural scenes, 10-class classification). For each specific application, an identical neural network architecture is engineered across three independent implementation paradigms: pure vectorized NumPy developed from basic principles without deep learning libraries, declarative TensorFlow/Keras, and modular PyTorch. The objective of this assignment is not to optimize nine competitive candidates for benchmark leaderboards. Instead, it directly examines the core hypothesis articulated in Lecture 04: an identical learning formulation, realized through an identical network architecture, will yield near-identical empirical results regardless of software abstraction tier. What genuinely diverges among these abstraction levels is algorithmic visibility, development effort, and wall-clock execution throughput, rather than the underlying mathematical modeling capability.

Table 1 outlines the three experimental applications and identifies which implementation achieved the top test classification accuracy, which serves as the primary benchmark metric for this study.

Table 1: Executive summary across the three applications

Application	Data type	Task	Architecture	Winning implementation
Diabetes	tabular	binary classification	MLP ($50 \to 32 \to 16 \to 1$)	PyTorch (0.782)
Fashion-MNIST	image, $28 \times 28 \times 1$	10-class classification	CNN (421,642 params)	Scratch (0.897)
CIFAR-10	image, $32 \times 32 \times 3$	10-class classification	CNN (545,098 params)	Keras (0.610)

Two primary takeaways stand out prior to examining the detailed per-application sections. First, the total trainable parameter count within every application matches bit-for-bit across all three implementations, verified through programmatic assertions in each notebook (`assert keras_param_count == scratch_param_count`, alongside the corresponding equality assertion against PyTorch). This establishes the foundational validation of our setup: the three codebases genuinely realize the exact same architecture, rather than three divergent models sharing a superficial moniker. Second, the top-performing framework alternates across applications: PyTorch leads on the diabetes tabular task, Scratch achieves the highest accuracy on Fashion-MNIST, and Keras emerges ahead on CIFAR-10. In all three cases, the margin separating the top-performing framework from the lowest is modest, spanning at most a few percentage points. This shifting rank order, combined with narrow metric spreads, directly substantiates Lecture 04: if a specific library possessed an innate mathematical or modeling advantage, it would dominate uniformly across all tasks. Instead, an alternating hierarchy is precisely what three functionally equivalent implementations of a shared model ought to produce, with small differences governed by framework-level initialization seeds and batch shuffling permutations rather than learning capacity. Training duration, however, reveals an unambiguous dynamic: Keras is consistently the fastest implementation across all domains, whereas the custom NumPy implementation incurs severe computational overhead specifically on the two convolutional networks. This exposes the concrete execution penalty of complete algorithmic visibility when performing uncompiled spatial tensor operations, a trade-off that the compact diabetes MLP (whose matrix operations are small enough that loop overhead remains negligible) does not show.

2 Theoretical Foundations: Architectural Invariance vs. Abstraction Layers

This section formalizes the overarching framework governing all three problem settings, establishing the conceptual foundation once so that subsequent application sections do not need to re-derive common theoretical arguments. Each per-application section that follows grounds these principles in concrete source code and empirical measurements, cross-referencing this section rather than reiterating it.

2.1 The Core Inquiry: Invariance across Software Abstractions

Lecture 04 frames a foundational inquiry that motivates this entire project: when building three implementations of an identical convolutional neural network across NumPy from scratch, TensorFlow/Keras, and PyTorch, *what remains invariant, and what diverges?* The lecture's direct answer is that the underlying conceptual architecture remains strictly invariant across all three tiers; what diverges is solely the abstraction level through which operations are expressed. In the custom NumPy codebase, a 2D convolution is an explicit procedural function performing an `im2col` matrix multiplication, readable and inspectable line by line. The identical operation in Keras is declared in a single line, `Conv2D(...)`, with the spatial sliding, multiply-accumulate steps, and memory layouts encapsulated inside compiled library kernels. The same operation in PyTorch, `nn.Conv2d(...)`, occupies a deliberate midpoint: the layer itself is instantiated as a compact modular block, yet the training loop that executes it, the gradient reset, and the backward propagation step remain completely explicit in user code rather than being absorbed into a monolithic fitting routine.

2.2 Conceptual Equivalence across Diverse Framework APIs

As emphasized in Lecture 04, syntactic differences and divergent library terminologies do not indicate different machine-learning concepts. Every implementation throughout this assignment translates the same seven structural operations (spatial convolution for image domains, nonlinear activation, spatial pooling for image domains, tensor flattening for image domains, affine linear projections, objective loss quantification, and parameter optimizer updates) under library-specific identifiers. Table 2 reproduces the concept-to-API correspondence introduced in the lecture, refined to the operations utilized in our experiments. Comprehensive, per-application tables detailing the precise syntax written in each codebase appear within their respective sections below, mapping the exact lines of code rather than generic placeholders.

Table 2: Concept-to-API correspondence used throughout this project

Concept	Purpose	Scratch	Keras / PyTorch
Convolution	local feature extraction	`conv2d_forward` (`im2col`)	`Conv2D` / `nn.Conv2d`
Activation	nonlinearity	`relu()`	`"relu"` / `nn.ReLU()`
Pooling	spatial reduction	`max_pool2d_forward`	`MaxPooling2D` / `nn.MaxPool2d`
Flatten	tensor $\rightarrow$ vector	reshape	`Flatten()` / `nn.Flatten()`
Dense/Linear	classification	`linear_forward` (matmul)	`Dense` / `nn.Linear`
Loss	measure error	manual formula	framework loss class
Optimizer	update parameters	`W -= lr * dW`	`optimizer="sgd"` / `torch.optim.SGD`

2.3 The Algorithmic Visibility and Developer Ergonomics Spectrum

The from-scratch NumPy implementation sits at the extreme of maximum visibility: every matrix dot product, analytical gradient derivation, and parameter modification is implemented by hand and can be inspected line by line. The trade-off is substantial development effort and, as demonstrated repeatedly in the empirical benchmarks below, significantly slower training throughput once multi-channel spatial convolutions are executed. Keras occupies the opposite pole of maximum convenience: the entire network is declared via a concise `Sequential([...])` block, and `model.fit(...)` encapsulates batch scheduling, forward propagation, loss evaluation, backward differentiation, and weight updates completely out of sight of user code. PyTorch is positioned deliberately in between: layer definitions are as declarative and modular as Keras within an `nn.Module` structure, but the core optimization cycle (`zero_grad` $\rightarrow$ forward pass $\rightarrow$ loss computation $\rightarrow$ `backward()` $\rightarrow$ `step()`) remains explicit within user Python code, utilizing reverse-mode automatic differentiation (`autograd`) strictly for computing gradients rather than hiding the surrounding loop.

2.4 Experimental Control and Fairness Protocol

Lecture 04 articulates the experimental condition required for these comparative evaluations to be scientifically sound: *same dataset, same split, same architecture, comparable hyperparameters*. Every application evaluated in this report adheres strictly to this protocol. Within each task, the data partition splits, layer dimensions and depths, mini-batch sizes, training epoch counts, and optimization learning rates are decided in advance and held completely constant across all three software implementations. The only area where complete bit-level parity is not enforced is initial weight initialization: the custom NumPy implementation utilizes explicit, seeded He normal initialization (`np.random.randn(...) * sqrt(2/fan_in)` with zero biases), whereas Keras and PyTorch employ their default layer initializers. While both framework defaults are also He/Kaiming variants designed for ReLU activations, they do not produce identical numeric starting weights due to distinct underlying pseudorandom generator sequences. This variation is treated as an authentic, observable factor contributing to minor empirical divergence in the reported results, rather than an error to be artificially masked.

3 Application I: Tabular Diabetes Metabolic Risk Screening (MLP)

3.1 Problem Formulation and Clinical Scope

This application predicts whether an individual surveyed by the Behavioral Risk Factor Surveillance System (BRFSS) has diagnosed diabetes or pre-diabetes, relying on self-reported demographic attributes and clinical health indicators. The input observation is an unstructured tabular vector with no spatial locality or 2D coordinate adjacency across columns: body mass index shares no local spatial relationship with marital status in the manner that neighboring pixels do within an image. Consequently, this task employs a fully connected feed-forward Multi-Layer Perceptron (MLP) rather than a CNN. This choice directly embodies Lecture 04's principle that *there is no natural convolution operation* for tabular feature collections of this structure.

3.2 Data Acquisition and Longitudinal Schema Assembly

The dataset integrates five annual data files from the Kaggle BRFSS indicator repository (https://www.kaggle.com/datasets/spandanjit2005/brfss-diabetes-indicator-dataset), spanning `DATASET_2016.csv` through `DATASET_2020.csv`. The tables were intersected on the 31 columns shared across all five years and concatenated into a unified table comprising **2,181,125 rows**. Two attributes, `high_bp_00` and `high_cholesterol_00`, appear only in the 2017 and 2019 survey questionnaires, reflecting questionnaire design rotations rather than random missing data. Both features were therefore excluded during schema intersection rather than retaining them with three years of imputed entries.

The target variable is derived from the BRFSS `diabetes` attribute, which originally recorded three categories (non-diabetic, pre-diabetic, and diabetic). In our formulation, this is collapsed into a binary label: 0 designates non-diabetic respondents, and 1 represents pre-diabetic and diabetic respondents combined. The resulting target exhibits notable class imbalance: 1,841,657 non-diabetic records against 339,468 diabetic or pre-diabetic records, corresponding to a 15.56% positive prevalence rate (Figure 1).

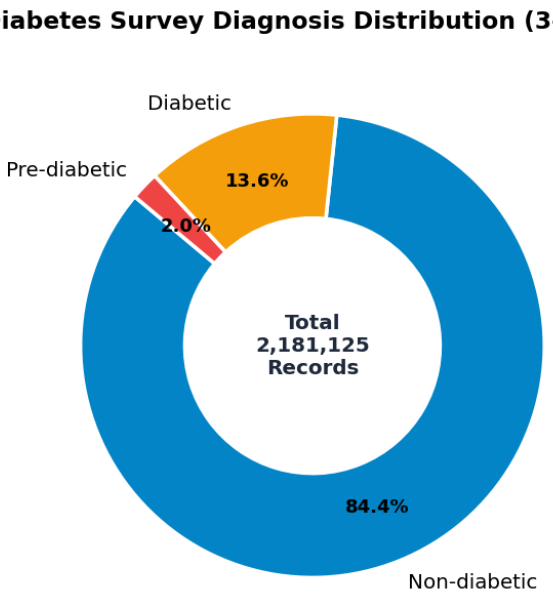


Figure 1: Donut chart illustrating the original 3-class distribution of diabetes survey responses prior to binary aggregation ($N = 2,181,125$ records), highlighting substantial class imbalance.

3.3 Exploratory Analysis, Sentinel Filtering, and Preprocessing

The BRFSS survey instrument employs specific numeric sentinel values to indicate ”refused to answer” or ”don’t know”, rather than leaving cells empty. These sentinel numbers can lie well outside the physical range of valid responses. Specifically, `weight_00` contains a large spike at exactly 999 pounds (accounting for 153,060 rows), an impossible real weight and an artificial round number that would not occur naturally in clinical data. Therefore, all rows with `weight_00 == 999` are converted to missing values rather than treated as real measurements. The same rationale applies to entries where `bmi_00 > 100` (174,083 rows), representing implausible body mass index values. Both sentinel categories are mapped to null values prior to the train/test split, with median imputation values computed strictly from the training partition to avoid data leakage.

Missing values in other attributes stem from questionnaire design logic: `avg_drinks_p_day_00` is unpopulated for 52.5% of respondents and `poor_health_days_00` is absent for 48.6%, because both are conditional questions presented only to individuals who indicated alcohol consumption or recent poor health. The `income_00` field is missing for 17.8% of participants, consistent with standard non-response rates on survey financial questions.

Several attributes proved to be string-formatted duplicates of numeric ordinal features already present in the data (`age_group_00` duplicate of `age_00`, `education_level_00` of `education_00`, `income_level_00` of `income_00`, `general_health_00` of `gen_health_00`, and `last_checkup_00` of `l_checkup_00`). Retaining both representations would either double-count identical signals or require arbitrary string re-encoding. Consequently, the five redundant string columns were dropped while their corresponding numeric ordinal versions were preserved.

Exploratory analysis confirmed key clinical associations that the model is expected to capture. Median BMI increases progressively from non-diabetic to pre-diabetic and diabetic respondents (Figure 2), consistent with

the established medical link between elevated adiposity and diabetes risk. Self-rated general health exhibits a strong monotonic relationship with diabetes status (Figure 3): participants rating their health as ”excellent” report diabetes or pre-diabetes at a much lower rate than those rating it ”poor”. For this reason, general health is maintained as a scaled numeric code rather than one-hot encoded, preserving the underlying ordinal signal. The diabetes prevalence across survey years (Figure 4) drifts mildly across the five-year period, ranging between 15.20% (2020) and 15.97% (2018). While this drift is subtle, survey `year` is preserved as a feature because it carries longitudinal survey-wave context following the omission of `high_bp_00` and `high_cholesterol_00`.

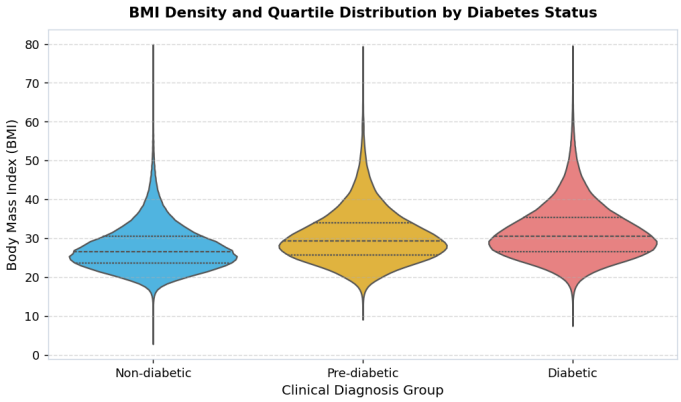


Figure 2: Violin plot depicting the density and quartile distribution of Body Mass Index (BMI) across clinical diagnosis groups, demonstrating consistent positive median shifts with disease progression.

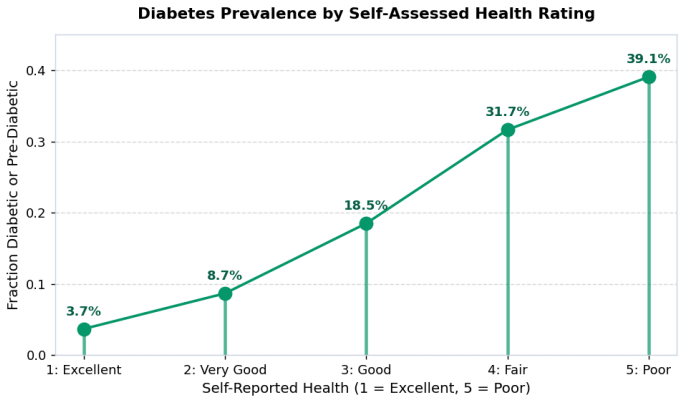


Figure 3: Connected lollipop trend plot showing diabetes prevalence as a function of self-assessed general health rating, revealing a monotonic escalation from Excellent (rating 1) to Poor (rating 5).

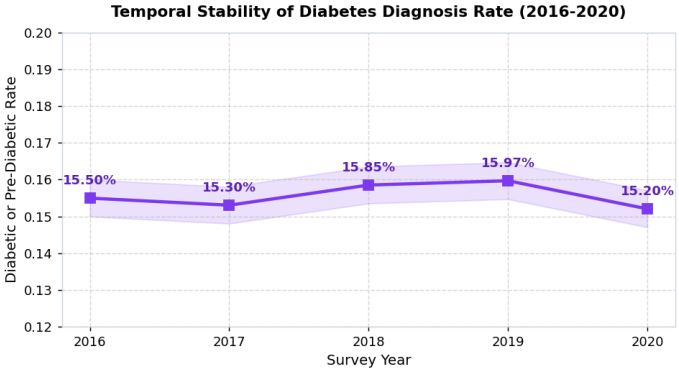


Figure 4: Temporal stability line plot tracking diabetes prevalence across survey waves (2016-2020), demonstrating consistent diagnosis rates across the multi-year corpus.

3.4 Feature Vector Encoding and Imbalance Mitigation

The remaining variables are organized into three transformation groups. Eight binary attributes (sex, physical activity, smoking history, alcohol use, medical history of stroke, heart attack, or coronary disease, and cost barriers to healthcare) map directly to $\{0, 1\}$ without information loss. Four nominal categorical attributes without natural ordering (race, marital status, employment status, and access to a personal physician) are one-hot encoded into 29 dummy columns, preventing artificial ordinal assumptions between distinct categories. The remaining 13 numeric attributes, consisting of ordinal survey ratings (general health, education level) alongside continuous measurements (BMI, weight, height) and survey `year`, are processed through a single `StandardScaler` fit strictly on the training partition.

The inclusion of `year` in the standardized numeric group is deliberate. Leaving calendar years unscaled (values from 2016 to 2020) would place this feature on a scale orders of magnitude larger than normalized binary flags, causing it to dominate initial affine projections and bias early gradient steps toward the majority class. Standardizing `year` prevents numerical dominance while retaining longitudinal wave signal.

This transformation yields an encoded feature matrix of width **50** (8 binary + 29 one-hot + 13 standardized numeric). An 80/20 stratified split (`RANDOM_SEED = 42`) produces 1,744,900 training records and 436,225 test records, preserving the 15.56% positive prevalence across both subsets.

To prevent optimization from collapsing to trivial majority-class prediction under class imbalance, balanced sample weights are computed as $w = n_{\text{total}}/(2 \times n_{\text{class}})$, resulting in $w_{\text{neg}} = 0.5922$ and $w_{\text{pos}} = 3.2126$. Under this formulation, misclassifying a diabetic instance incurs a loss penalty roughly 5.4 times greater than misclassifying a non-diabetic instance. All three implementations incorporate this balanced weighting, though their underlying execution mechanics diverge as discussed below.

3.5 Unified Multi-Layer Perceptron Topology

All three implementations construct the identical Multi-Layer Perceptron determined dynamically from the verified input width $d = 50$:

$$50 \to \text{Linear}(32) \to \text{ReLU} \to \text{Linear}(16) \to \text{ReLU} \to \text{Linear}(1) \to \text{Sigmoid}$$

with the following layer shape trace for a batch of size B:

$$(B, 50) \to (B, 32) \to (B, 32) \to (B, 16) \to (B, 16) \to (B, 1) \to (B, 1)$$

A mini-batch size of 2048, 25 training epochs, a learning rate of 0.1, and plain vanilla SGD (without momentum or adaptive moments) are established once and reused across all three codebases, ensuring that the underlying optimization updates are identical.

3.6 Implementation Across Software Paradigms

3.6.1 Vectorized NumPy Implementation from First Principles

All functional components are written as explicit Python functions: `relu()`, a custom `binary_cross_entropy(y_true, y_pred, weights=None)` supporting sample weighting, He-initialized weight matrices (`init_params`), an explicit `forward` pass chaining two `Linear + ReLU` layers into a final `Linear + Sigmoid`, and an analytical `backward` pass derived via the chain rule. The balanced sample weights are multiplied directly into the output error gradient $dZ_3 = (A_3 - y) \odot w$, propagating the class penalty through every upstream weight gradient. A finite-difference numerical gradient check confirmed backward pass correctness prior to full training. Mini-batch gradient descent (batch size 2048) executes for 25 epochs with manual parameter updates.

3.6.2 Declarative TensorFlow/Keras Implementation

The identical architecture is declared as a sequential model combining `Dense(32, "relu")`, `Dense(16, "relu")`, and `Dense(1, "sigmoid")`. It is compiled with `SGD(learning_rate=0.1)` and binary cross-entropy loss, and trained via `model.fit(..., class_weight={0: 0.5922, 1: 3.2126})`. Keras's `class_weight` multiplies each class's loss contribution by the same relative factors as the custom implementation, training on an identically weighted objective. Programmatic inspection via `model.summary()` confirms exactly 2,177 parameters, matching the NumPy baseline bit-for-bit.

3.6.3 Modular PyTorch Implementation

The model is defined as an `nn.Module` subclass (`DiabetesMLP`) whose `forward` method chains three `nn.Linear` projections separated by `nn.ReLU`, returning unnormalized log-odds (logits) without an internal sigmoid activation. The loss is computed via `nn.BCEWithLogitsLoss(pos_weight=...)`, combining sigmoid evaluation and cross-entropy into a single numerically stable kernel. The positive weight parameter is set to $w_{\text{pos}}/w_{\text{neg}} \approx 5.42$, PyTorch's native formulation for balanced binary weighting. The optimization loop executes explicit iterations of `zero_grad` $\rightarrow$ forward pass $\rightarrow$ `loss.backward()` $\rightarrow$ `optimizer.step()` using `torch.optim.SGD`. The resulting model totals 2,177 trainable parameters, matching the other two implementations.

3.7 Structural and Operational Framework Mapping

Table 3: Diabetes: component-by-implementation table

Component	Scratch	Keras	PyTorch
Data loading	`pandas.read_csv` ×5, `pd.concat`	same	same
Preprocessing	manual mean/std standardization	StandardScaler-equivalent (manual, shared)	same as scratch
Model definition	explicit functions + a params dict	`Sequential([...])`	`nn.Module` subclass
Activation	`relu()`, `sigmoid()`	`activation="relu"/"sigmoid"`	`nn.ReLU()`, fused in `BCEWithLogitsLoss`
Dense/Linear layer	`linear()` (manual matmul)	`Dense(...)`	`nn.Linear(...)`
Loss	manual `binary_cross_entropy`	`"binary_crossentropy"`	`nn.BCEWithLogitsLoss()`
Gradient	manual backprop	autodiff (implicit in `fit`)	`loss.backward()` (autograd)
Optimizer	manual `W -= lr * dW`	`optimizer="sgd"`	`torch.optim.SGD(...)`
Training loop	explicit `for epoch / for batch`	`model.fit(...)`	explicit `for epoch / for batch`
Evaluation	manual metric functions	`model.evaluate`-equivalent (manual)	manual (`model.eval()`, no-grad)
Prediction	`forward(X_test, params)`	`model.predict(...)`	`model(x_test)` under `torch.no_grad()`

3.8 Empirical Optimization Dynamics and Evaluation

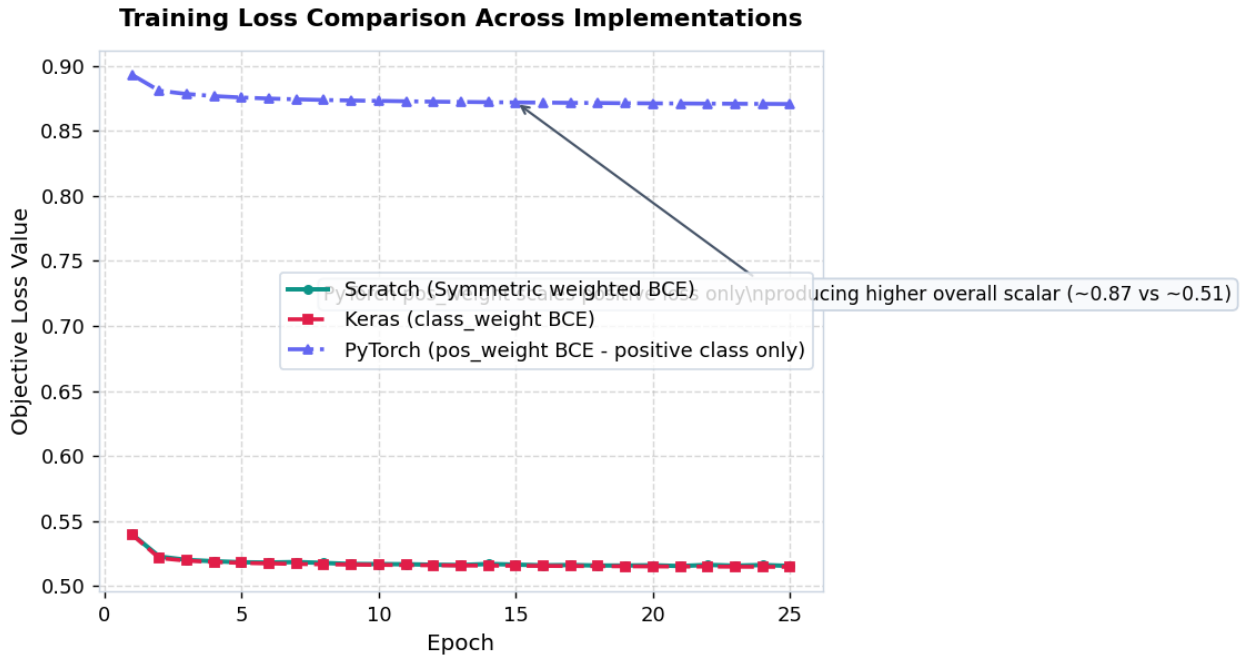


Figure 5: Comparative training loss curves across implementations, highlighting the numeric scale discrepancy resulting from PyTorch's asymmetric positive-class loss weighting.

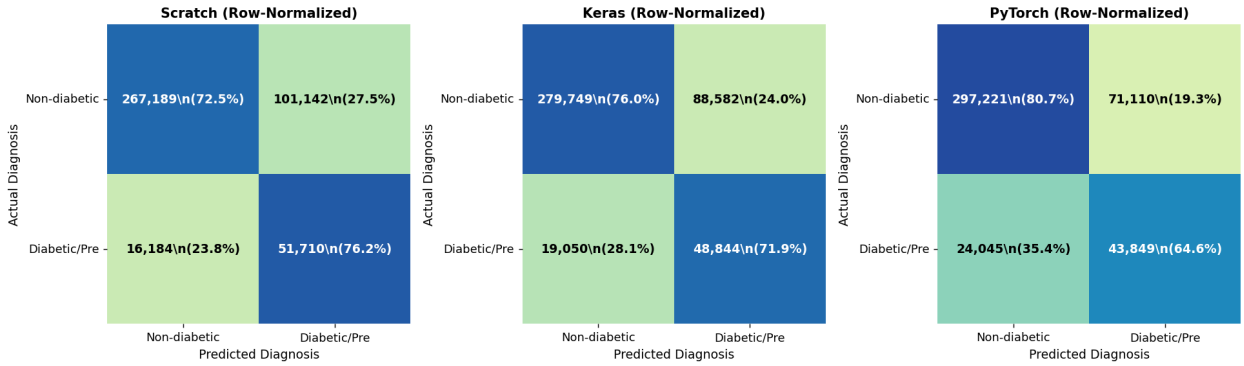


Figure 6: Row-normalized confusion matrices with cell counts and recall percentages across Scratch, Keras, and PyTorch on the diabetes test split ($N = 436,225$).

Table 4: Diabetes: quantitative comparison, test set

Implementation	Params	Train time (s)	Final loss	Accuracy	Precision	Recall
Scratch	2,177	69.4	0.5155	0.7310	0.3383	0.7616
Keras	2,177	24.7	0.5148	0.7533	0.3554	0.7194
PyTorch	2,177	311.0	0.8706	0.7819	0.3814	0.6458

F1 scores for the three implementations in the same order are: 0.4685 (Scratch), 0.4758 (Keras), and 0.4796 (PyTorch).

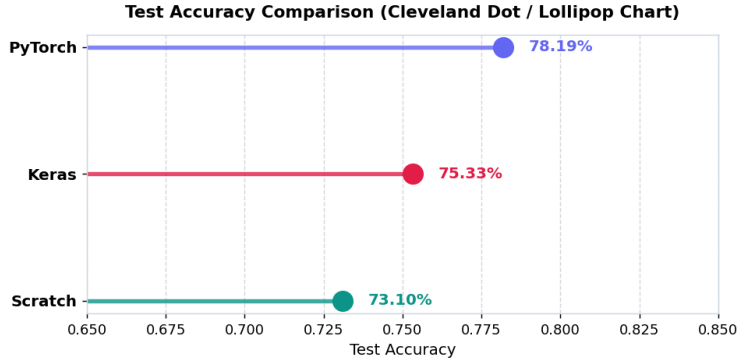


Figure 7: Cleveland dot / horizontal lollipop chart comparing test classification accuracy across implementations on the diabetes benchmark.

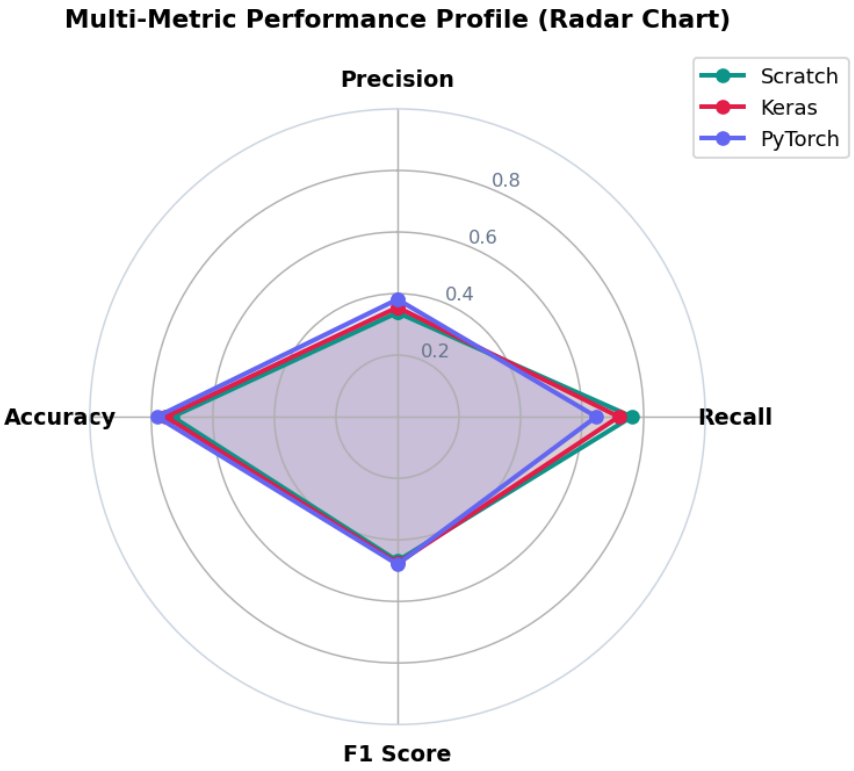


Figure 8: Multi-metric radar chart evaluating Precision, Recall, F1 score, and Accuracy profiles across Scratch, Keras, and PyTorch implementations.

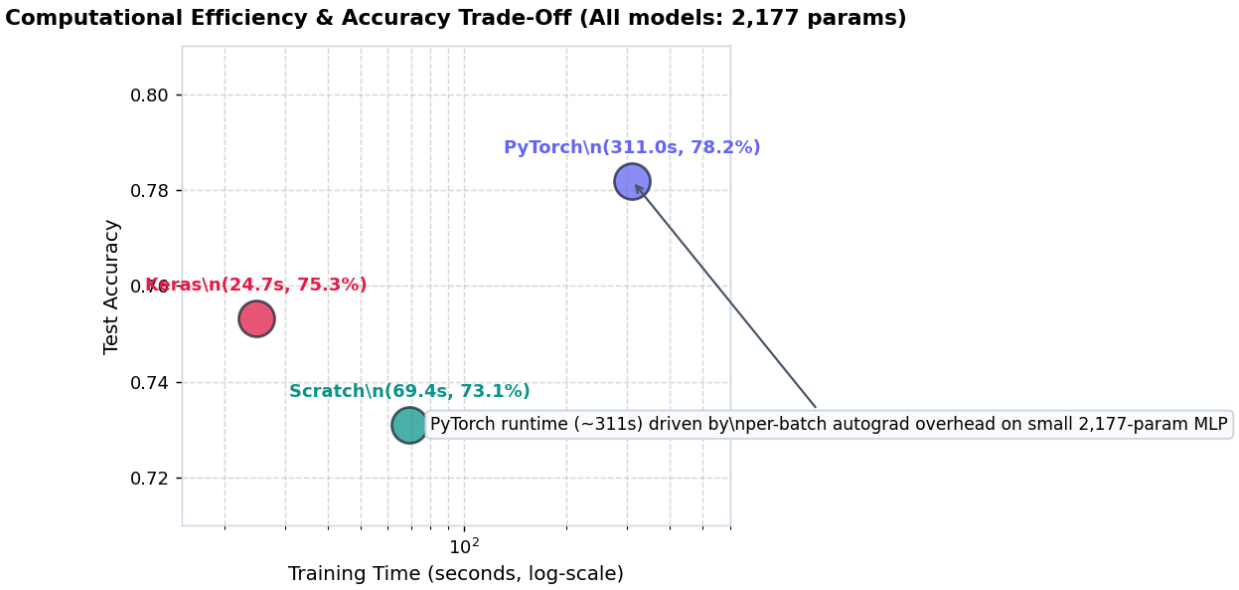


Figure 9: 2D Pareto efficiency bubble chart illustrating the trade-off between training duration (log scale) and test accuracy, with bubble dimensions scaled to parameter count (2,177 parameters across all models).

3.9 Comparative Analysis and Architectural Trade-Offs

Measured by test accuracy and F1 score, PyTorch achieves a slight advantage (0.782, 0.480), followed closely by Keras (0.753, 0.476) and Scratch (0.731, 0.469). All three models finish within approximately five percentage points of each other across both metrics. This tight clustering is the expected outcome for three implementations of an identical architecture trained on identical partitions using equivalent hyperparameters: the margin is small enough to be explained by framework-level nuances (default numerical precision, batch shuffling order, and minor differences in loss weighting parameterization) rather than fundamental modeling superiority.

One value in Table 4 is not directly comparable across frameworks, and warrants explicit explanation: the final training loss. Scratch and Keras both converge near 0.51, whereas PyTorch records a loss of 0.8706, a difference substantially larger than the accuracy spread would suggest. This divergence stems entirely from how each framework formulates balanced class weighting, rather than inferior optimization in PyTorch. Scratch and Keras both apply symmetric sample weighting, multiplying negative instance losses by $w_{\text{neg}} = 0.5922$ and positive instance losses by $w_{\text{pos}} = 3.2126$, maintaining overall loss magnitude on roughly the same scale as unweighted cross-entropy. In contrast, PyTorch's `BCEWithLogitsLoss(pos_weight=...)` scales only the positive class loss term by the ratio $w_{\text{pos}}/w_{\text{neg}} \approx 5.42$, leaving negative class loss terms unscaled. While this represents a mathematically valid formulation that enforces identical relative error trade-offs, it produces an algebraically higher absolute loss value. This finding illustrates how similar feature descriptions across frameworks (`class_weight`, custom sample weights, and `pos_weight`) can mask distinct mechanical implementations.

Training execution time presents a very different picture. Keras was the fastest framework (24.7s), Scratch was intermediate (69.4s), and PyTorch was noticeably slower (311.0s), requiring nearly four and a half times longer than Scratch for a network with only 2,177 parameters. This corresponds directly to execution overhead: Keras executes its training loop within a compiled computation graph with minimal per-batch Python overhead; Scratch uses an interpreted Python loop, but each batch involves only a few matrix multiplications on a tiny array, allowing raw NumPy C-extensions to dominate; PyTorch constructs and tears down dynamic autograd tape graphs at every step, incurring a fixed overhead that cannot be amortized across such a small parameter space. PyTorch's architectural flexibility (an explicit, inspectable Python loop) comes with concrete per-step overhead that is prominent on small networks, a dynamic that changes completely once convolutional operations are introduced in Sections 4 and 5.

Regarding transparency and control: Keras abstracts nearly all operational details, packaging loss calculations, weighting mechanics, backward propagation, and batch loops inside `compile()` and `fit()`. PyTorch abstracts only the backward gradient calculation, keeping training loops and loss weighting explicit, which directly exposed the `pos_weight` numeric scaling asymmetry. The custom NumPy implementation abstracts nothing, at the cost of requiring the most development and verification effort.

4 Application II: Fashion-MNIST Grayscale Apparel Categorization (CNN)

4.1 Problem Formulation and Spatial Inductive Bias

This application categorizes 28×28 grayscale images of apparel into one of ten garment classes (T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot). Unlike the diabetes tabular data, visual image inputs possess genuine 2D spatial coherence: neighboring pixels share semantic relationships, and visual features such as sleeves or shoe soles represent local geometric patterns that recur across different spatial locations. This establishes the exact scenario presented in Lecture 04 for employing a CNN rather than an MLP: local connectivity and parameter sharing allow a compact set of learned convolutional filters to detect translationally invariant visual patterns across the entire image grid.

4.2 Dataset Characterization and Partition Uniformity

Fashion-MNIST is retrieved via `keras.datasets.fashion_mnist.load_data()`, downloading and caching the benchmark files automatically. The dataset provides 60,000 training images and 10,000 test images, each formatted as a 28×28 single-channel array with raw integer pixel intensities in $[0, 255]$. Each of the ten classes contains exactly 6,000 training instances and 1,000 test instances (Figure 10). Because the classes are perfectly balanced, no class reweighting adjustments are necessary: a trivial majority classifier would achieve only 10% accuracy, preventing misleading performance inflation.

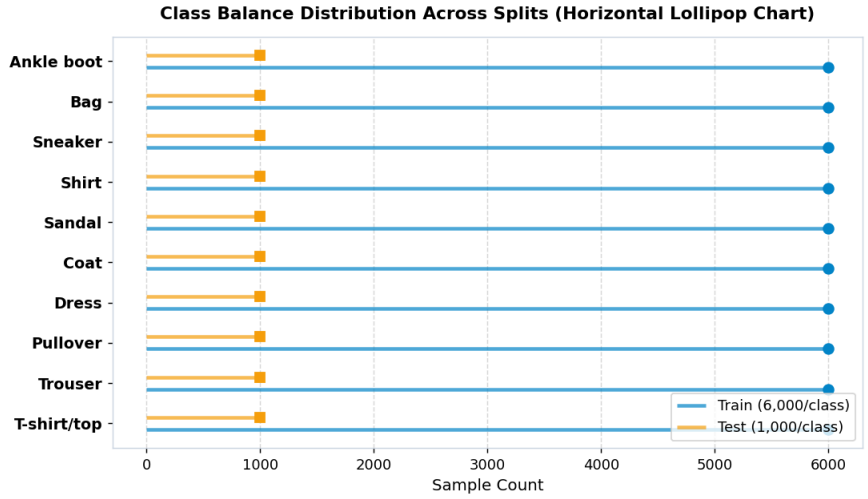


Figure 10: Horizontal lollipop chart illustrating balanced class distributions across training (6,000 samples/class) and test (1,000 samples/class) partitions for Fashion-MNIST.

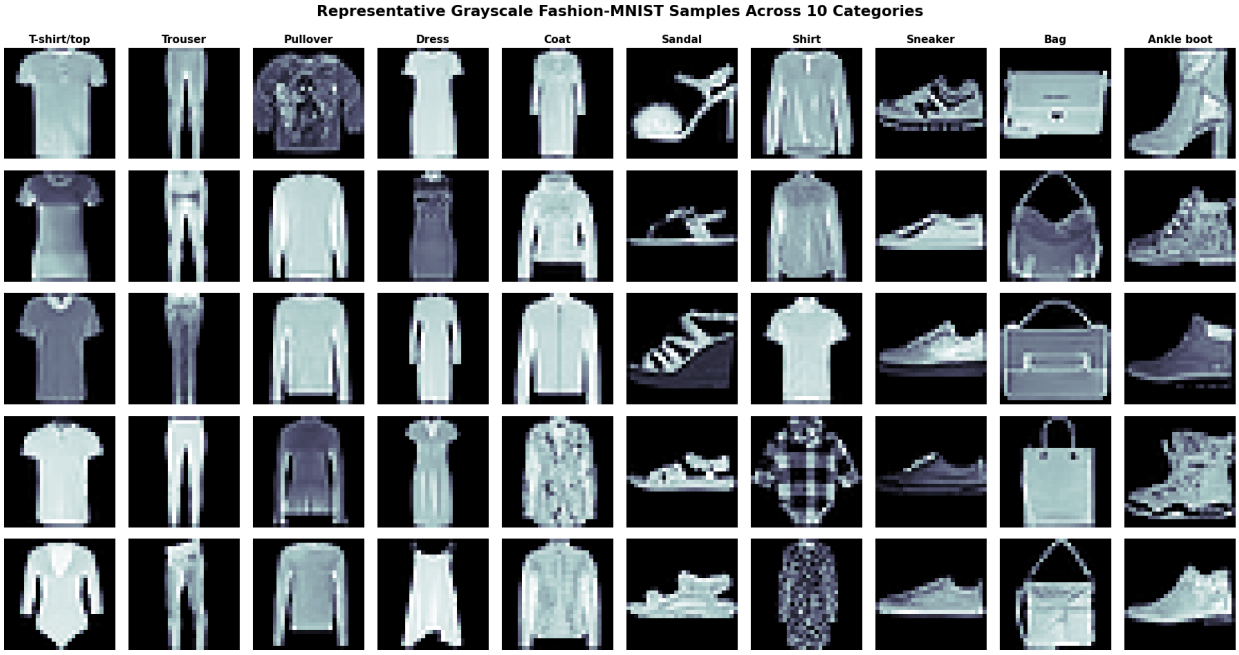


Figure 11: Representative 28×28 grayscale Fashion-MNIST garment samples across all ten categories, illustrating subtle intra-category silhouette overlap among upper-body apparel.

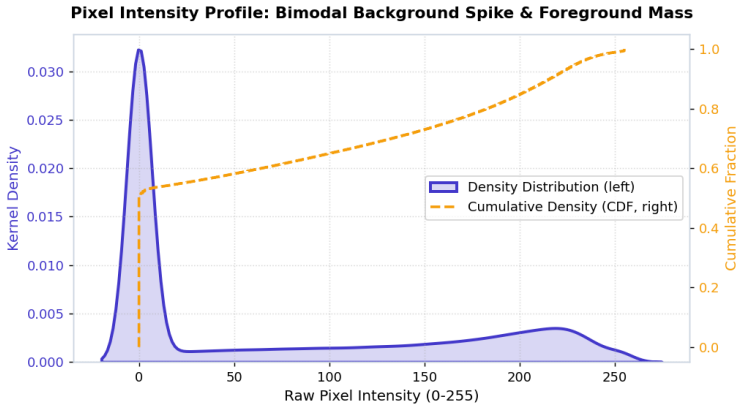


Figure 12: Dual-axis density profile and cumulative distribution function (CDF) of raw pixel intensities, showing the dominant background zero-spike and broad foreground fabric distribution.

4.3 Tensor Normalization and Memory Layout Alignment

Pixel intensities are converted to `float32` and divided by 255.0, mapping values from $[0, 255]$ into $[0, 1]$. Two tensor layouts are maintained: a channel-first layout $(N, 1, 28, 28)$ for the NumPy and PyTorch implementations, and a channel-last layout $(N, 28, 28, 1)$ for Keras's default convolution format. No additional data cleaning or imputation is needed, as Fashion-MNIST is a curated benchmark without missing entries or schema inconsistencies.

4.4 Unified Two-Stage Convolutional Topology

All three implementations build the identical two-stage convolutional architecture:

$$28 \times 28 \times 1 \to \text{Conv}(1{\to}32, k{=}3, \text{pad}{=}1) \to \text{ReLU} \to \text{MaxPool}(2) \to \text{Conv}(32{\to}64, k{=}3, \text{pad}{=}1) \to \text{ReLU} \to \text{MaxPool}(2)$$

$$\to \text{Flatten}(64{\times}7{\times}7 = 3136) \to \text{Linear}(3136{\to}128) \to \text{ReLU} \to \text{Linear}(128{\to}10)$$

with layer shape trace for a batch of size B:

$$(B, 1, 28, 28) \to (B, 32, 28, 28) \to (B, 32, 14, 14) \to (B, 64, 14, 14) \to (B, 64, 7, 7) \to (B, 3136) \to (B, 128) \to (B, 10)$$

A mini-batch size of 256, 10 training epochs, a learning rate of 0.05, and vanilla SGD optimization are held constant across all implementations.

4.5 Implementation Across Software Paradigms

4.5.1 Vectorized im2col Convolutional Engine in NumPy

Every functional stage (convolutions, pooling, dense projections, loss evaluation, and gradients) is programmed from first principles. For the 2D spatial convolution, a 3×3 filter slides across input patches, computing dot products that populate output feature maps. Because nested loops across spatial coordinates were impractically slow (the second convolutional layer required over two seconds per mini-batch), the custom `conv2d_forward` and `conv2d_backward` routines utilize the classical `im2col` algorithm. Receptive field patches are extracted as zero-copy strided views and rearranged into a matrix, transforming spatial convolution into a single general matrix multiplication. This yields an execution speedup of approximately 40 times over naive nested loops. Analytical gradients across convolution and pooling layers were verified against finite-difference numerical approximations. This gradient verification detected an implementation defect during development: an extraneous $1/N$ division in the dense-layer backward pass was silently attenuating dense weight gradients. Correcting this restored proper convergence before final training. Optimization runs via mini-batch SGD (batch size 256, 235 iterations per epoch) over 10 epochs.

4.5.2 Declarative TensorFlow/Keras Implementation

The identical architecture is declared as an eight-layer `Sequential` model: `Conv2D(32, 3, padding="same", activation="relu")`, `MaxPooling2D(2)`, `Conv2D(64, 3, padding="same", activation="relu")`, `MaxPooling2D(2)`, `Flatten()`, `Dense(128, "relu")`, and `Dense(10)`. Compilation specifies `SGD(learning_rate=0.05)` and `SparseCategoricalCrossentropy(from_logits=True)`, trained via `model.fit(..., batch_size=256, epochs=10)`. Programmatic auditing confirms 421,642 trainable parameters, matching the NumPy baseline exactly. The explicit looping and memory manipulations written in Scratch are encapsulated within Keras layer abstractions.

4.5.3 Modular PyTorch Implementation

The model is structured as an `nn.Module` subclass (`FashionCNN`) comprising a `self.features` container (`nn.Conv2d`, `nn.ReLU`, `nn.MaxPool2d` repeated twice) followed by a `self.classifier` container (`nn.Flatten`, `nn.Linear(3136, 128)`, `nn.ReLU`, `nn.Linear(128, 10)`), outputting unnormalized logits for `nn.CrossEntropyLoss()`. The training loop executes explicit iterations of `zero_grad` $\to$ forward pass $\to$ loss computation $\to$ `backward()` $\to$ `step()` using `torch.optim.SGD`. Total parameters equal 421,642, confirmed via assertions in the notebook.

4.6 Structural and Operational Framework Mapping

Table 5: Fashion-MNIST: component-by-implementation table

Component	Scratch	Keras	PyTorch
Data loading	Keras loader, shared NumPy arrays	same	same
Preprocessing	manual /255, channel-first reshape	/255, channel-last (default)	/255, channel-first
Model definition	explicit functions + a params dict	`Sequential([...])`	`nn.Module` subclass
Convolution	`conv2d_forward/backward` (`im2col`)	`Conv2D(...)`	`nn.Conv2d(...)`
Activation	`relu()`	`activation="relu"`	`nn.ReLU()`
Pooling	`max_pool2d_forward/backward`	`MaxPooling2D(2)`	`nn.MaxPool2d(2)`
Flatten	`flatten_forward/backward` (reshape)	`Flatten()`	`nn.Flatten()`
Dense/Linear layer	`linear_forward/backward`	`Dense(...)`	`nn.Linear(...)`
Loss	manual softmax cross-entropy	`SparseCategorical Crossentropy( from_logits=True)`	`nn.CrossEntropyLoss()`
Gradient	manual backprop	autodiff (implicit in `fit`)	`loss.backward()` (autograd)
Optimizer	manual `W -= lr * dW`	`optimizer="sgd"`	`torch.optim.SGD(...)`
Training loop	explicit `for epoch / for batch`	`model.fit(...)`	explicit `for epoch / for batch`
Evaluation	manual metric functions	`model.evaluate`-equivalent (manual)	manual (`model.eval()`, no-grad)
Prediction	`forward(X_test, params)`	`model.predict(...)`	`model(x_test)` under `torch.no_grad()`

4.7 Empirical Optimization Dynamics and Evaluation

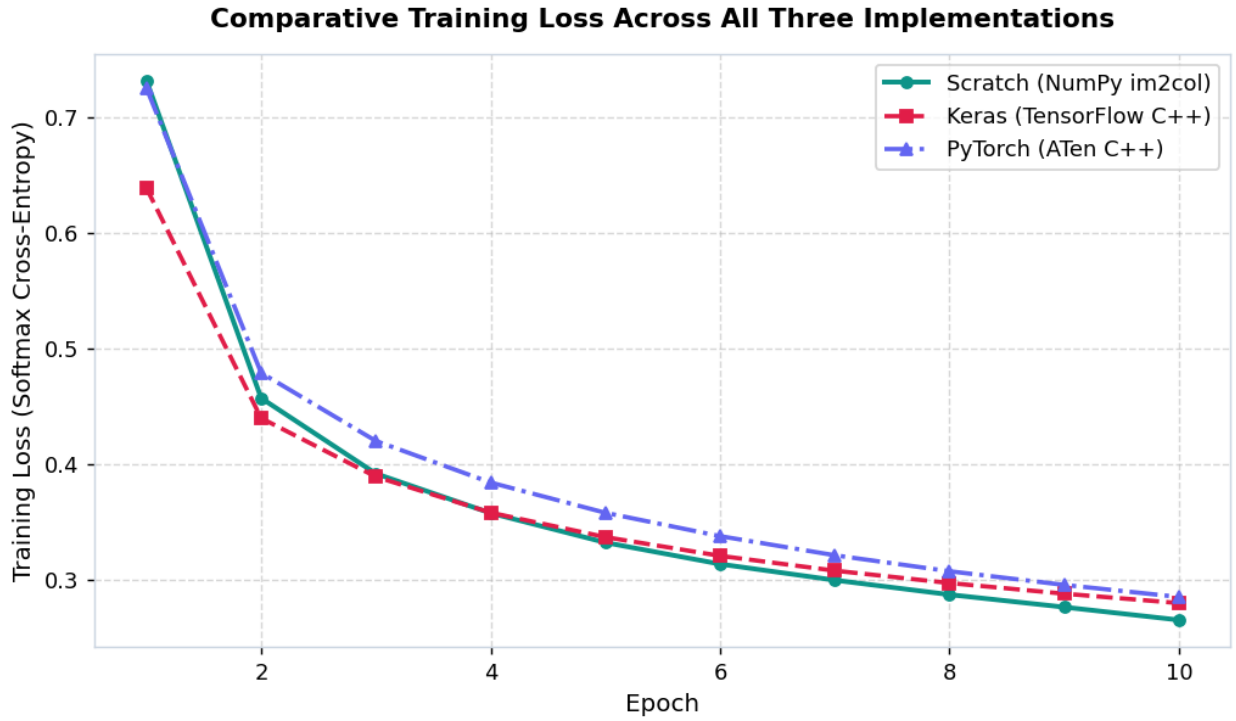


Figure 13: Comparative training loss trajectories across the three implementations on Fashion-MNIST over 10 training epochs.

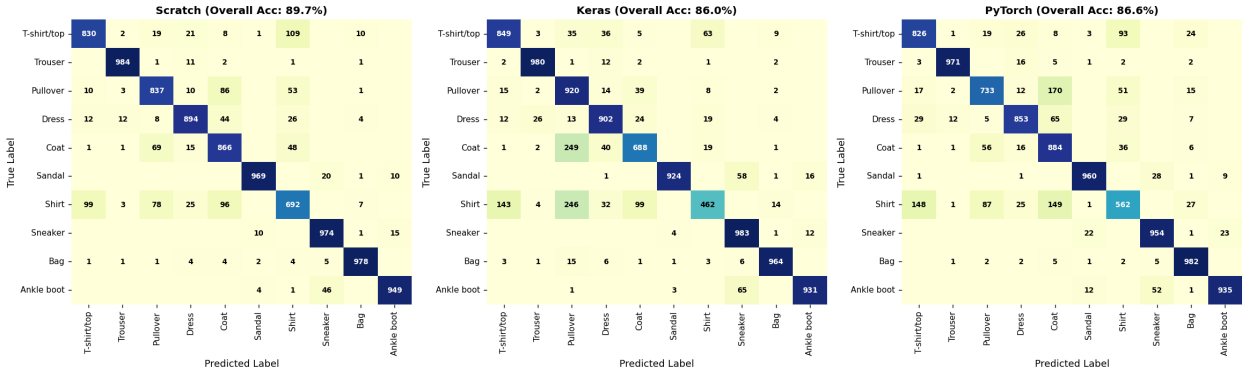


Figure 14: Normalized confusion matrices across Scratch, Keras, and PyTorch on Fashion-MNIST ($N = 10,000$), demonstrating consistent off-diagonal confusion concentrated within the shirt, pullover, and coat cluster.

Table 6: Fashion-MNIST: quantitative comparison, test set

Implementation	Params	Train time (s)	Final loss	Accuracy	Precision (macro)	Recall (macro)
Scratch	421,642	1,330.5	0.2655	0.8973	0.8976	0.8973
Keras	421,642	75.6	0.3157	0.8603	0.8695	0.8603
PyTorch	421,642	191.5	0.3544	0.8660	0.8676	0.8660

Macro F1 scores for the three implementations in the same order are: 0.8970 (Scratch), 0.8565 (Keras), and 0.8641 (PyTorch).

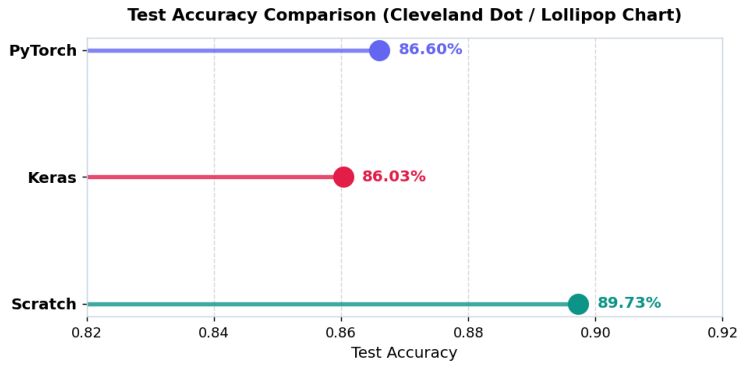


Figure 15: Cleveland dot / horizontal lollipop plot comparing test accuracy across implementations, highlighting the custom NumPy model's slight accuracy advantage.

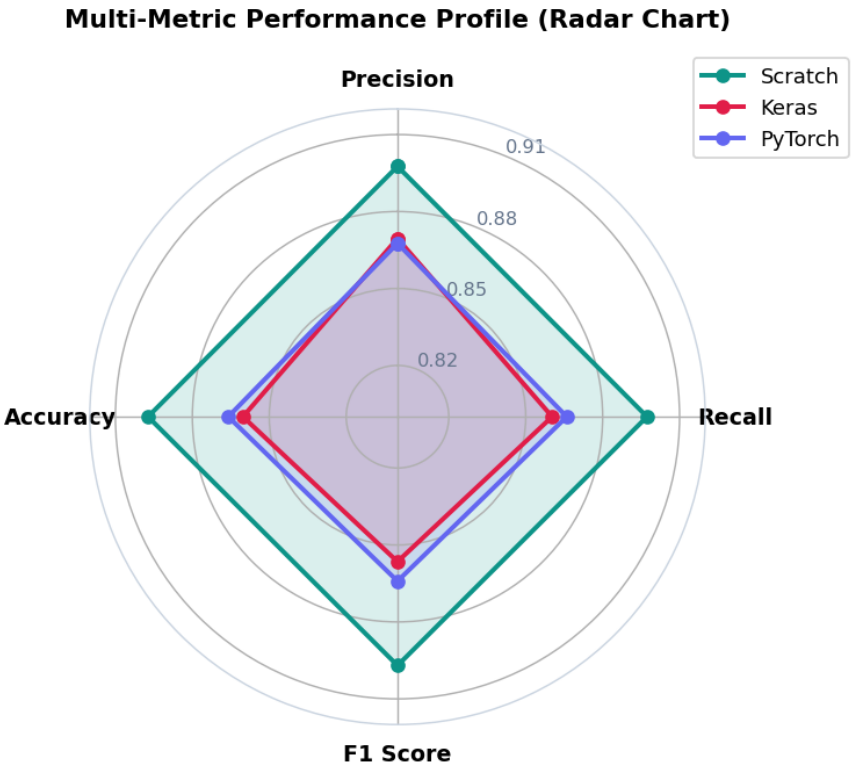


Figure 16: Multi-metric radar chart depicting macro-averaged Precision, Recall, F1 score, and Accuracy across all three implementations.

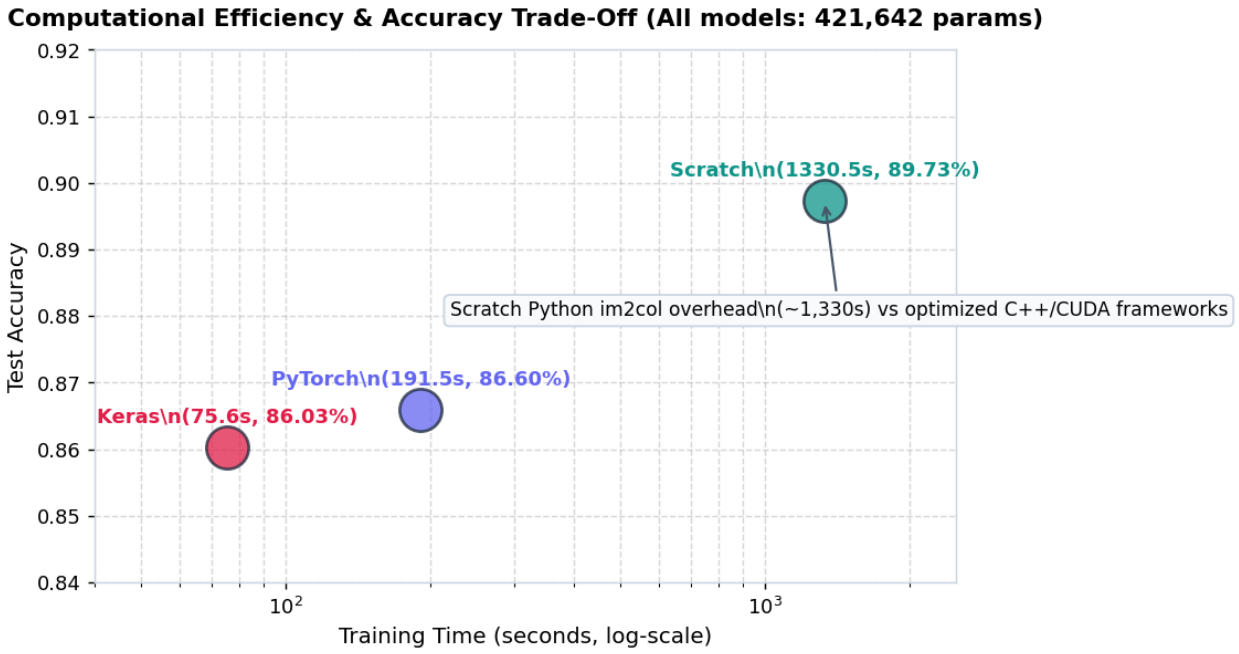


Figure 17: 2D Pareto efficiency trade-off chart plotting training runtime against test accuracy, demonstrating the significant wall-clock latency of uncompiled Python convolutions.

4.8 Comparative Analysis and Computational Scaling

In test classification accuracy, the custom NumPy implementation achieves the highest score: 0.897 compared to 0.866 for PyTorch and 0.860 for Keras. This is an intriguing empirical result, as from-scratch code is rarely expected to lead. The explanation lies in initialization variance: Scratch applies explicit He initialization with a fixed random seed, whereas Keras and PyTorch rely on their framework defaults, which, while also ReLU-appropriate, yield slightly different parameter trajectories. Over a compact 10-epoch schedule, initial trajectory advantages can persist in final accuracy scores. All three implementations remain within four percentage points of each other, consistent with three realizations of an identical model topology and echoing the metric spread observed in the diabetes application.

Training duration highlights a dramatic contrast. Scratch required 1,330.5 seconds (approximately 22 minutes), Keras took 75.6 seconds, and PyTorch required 191.5 seconds. Scratch was roughly 17.6 times slower than Keras and 6.9 times slower than PyTorch. This discrepancy directly reflects execution backends: Keras executes its training loop inside a compiled graph using optimized C++ kernels with zero per-batch Python overhead; PyTorch incurs minor Python loop and autograd dispatch overhead, but computes convolutions via highly optimized, compiled routines; Scratch's `im2col` convolution, while vastly superior to nested position loops, still issues NumPy matrix multiplications from an interpreted Python loop without operator fusion or specialized low-level optimizations. While `im2col` made training feasible, it cannot close the gap with purpose-built deep learning engines. This contrasts sharply with the diabetes MLP (Section 3.9), where Scratch was competitive with the frameworks and PyTorch was slowest: once convolutions are introduced, the computational performance landscape shifts completely.

In terms of framework abstraction: Keras concealed all convolution, pooling, differentiation, and optimization mechanics behind declarative calls. PyTorch abstracted gradient computation while preserving explicit loop control. Scratch exposed every calculation, gradient derivation, and parameter update directly. The price of complete algorithmic visibility on convolutional architectures is a substantial increase in wall-clock training duration.

5 Application III: CIFAR-10 Photographic Object Recognition (CNN)

5.1 Problem Formulation and Multi-Channel Visual Complexity

This application categorizes 32×32 RGB color photographs into one of ten natural object classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). It adopts the same convolutional network family as Fashion-MNIST to leverage 2D spatial locality and weight sharing, but handles three chromatic input channels rather than one. Furthermore, the task is considerably more challenging: distinguishing natural animals and vehicles (such as differentiating cats from dogs, or automobiles from trucks) in low-resolution photographic scenes involves subtle object boundaries and diverse background clutter, representing a more difficult visual problem than segmenting apparel silhouettes.

5.2 Dataset Characterization and Chromatic Diversity

CIFAR-10 is loaded via `keras.datasets.cifar10.load_data()`, which downloads and caches the 170MB archive on first use. The dataset contains 50,000 training images and 10,000 test images, each formatted as a 32×32 array with three color channels (RGB) and integer pixel values in $[0, 255]$. The ten classes are uniformly distributed, containing exactly 5,000 training images and 1,000 test images per category (Figure 18), requiring no class-imbalance adjustments.

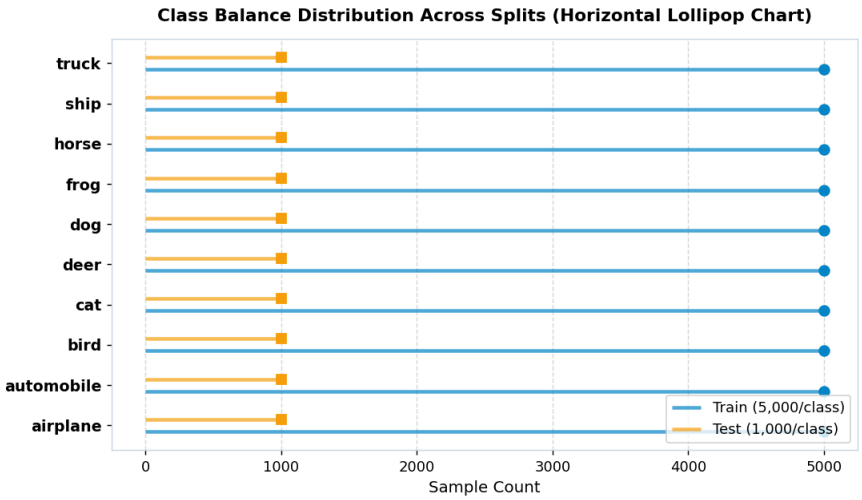


Figure 18: Horizontal lollipop chart confirming perfectly uniform class partitions across training (5,000 images/class) and test (1,000 images/class) splits on CIFAR-10.



Figure 19: Sample gallery of 32×32 RGB natural scene photographs across ten classes, showing chromatic diversity and fine-grained spatial ambiguity.

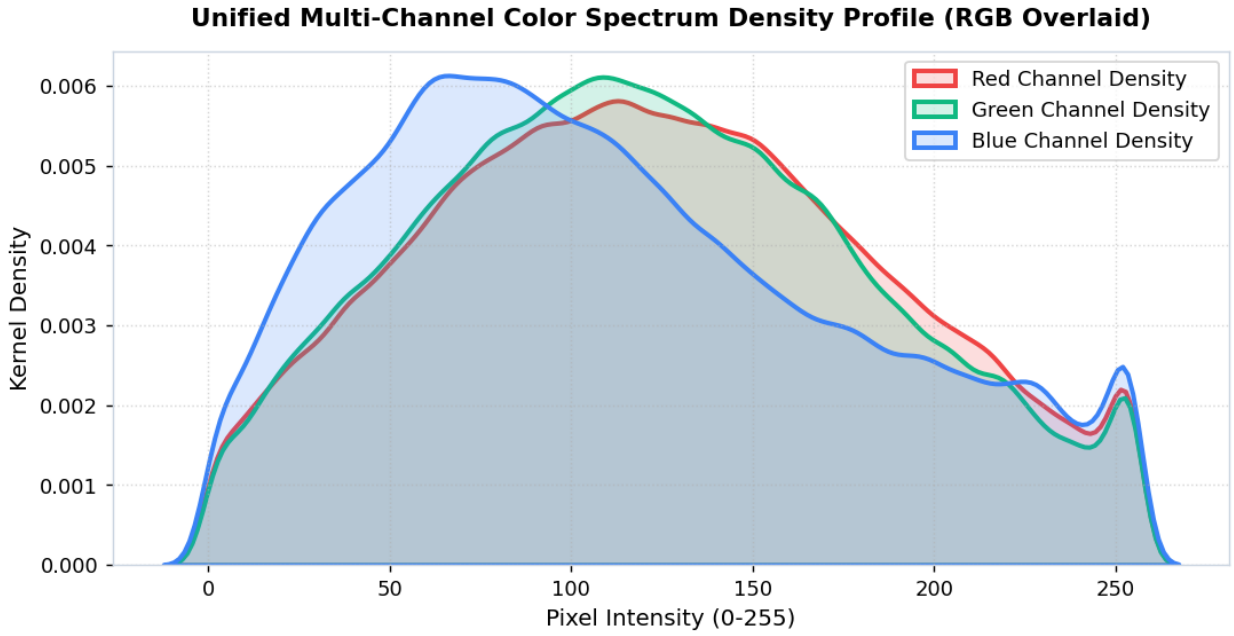


Figure 20: Unified overlaid multi-channel spectral density profile displaying kernel density estimates (KDE) across red, green, and blue color channels in a single coordinate system.

5.3 Multi-Channel Normalization and Layout Alignment

Pixel intensities are converted to `float32` and normalized to $[0, 1]$ via division by 255.0. CIFAR-10 is provided in channel-last orientation $(N, 32, 32, 3)$, which matches Keras's native format directly without array transposition. For Scratch and PyTorch, arrays are converted to channel-first orientation $(N, 3, 32, 32)$. No additional cleaning or imputation is necessary.

5.4 Unified Three-Channel Convolutional Topology

All three implementations construct the identical two-stage convolutional architecture adapted for 3-channel inputs:

$$3 \times 32 \times 32 \to \text{Conv}(3{\to}32, k{=}3, \text{pad}{=}1) \to \text{ReLU} \to \text{MaxPool}(2) \to \text{Conv}(32{\to}64, k{=}3, \text{pad}{=}1) \to \text{ReLU} \to \text{MaxPool}(2)$$

$$\to \text{Flatten}(64{\times}8{\times}8 = 4096) \to \text{Linear}(4096{\to}128) \to \text{ReLU} \to \text{Linear}(128{\to}10)$$

with layer shape trace for a batch of size B:

$$(B, 3, 32, 32) \to (B, 32, 32, 32) \to (B, 32, 16, 16) \to (B, 64, 16, 16) \to (B, 64, 8, 8) \to (B, 4096) \to (B, 128) \to (B, 10)$$

A mini-batch size of 256, 12 training epochs, a learning rate of 0.05, and vanilla SGD optimization are held constant across all implementations.

5.5 Implementation Across Software Paradigms

5.5.1 Multi-Channel im2col Convolutional Engine in NumPy

The scratch implementation adapts the `conv2d_forward/backward` (`im2col`), `max_pool2d_forward/backward`, `relu()`, and softmax cross-entropy routines from Fashion-MNIST to support three-channel inputs. The first convolutional kernel shape expands to $(32, 3, 3, 3)$ rather than $(32, 1, 3, 3)$ to accommodate RGB channels. A multi-channel numerical gradient check verified that backward passes remain exact for 3-channel tensors prior to full training. Training executes over 12 epochs (196 batches per epoch) using mini-batch SGD.

5.5.2 Declarative TensorFlow/Keras Implementation

The model is declared using Keras's sequential API, receiving $(32, 32, 3)$ inputs directly: `Conv2D(32, 3, padding="same", activation="relu")`, `MaxPooling2D(2)`, `Conv2D(64, 3, padding="same", activation="relu")`, `MaxPooling2D(2)`, `Flatten()`, `Dense(128, "relu")`, and `Dense(10)`. Compilation uses `SGD(learning_rate=0.05)` and `SparseCategoricalCrossentropy(from_logits=True)`, training for 12 epochs via `model.fit(..., batch_size=256)`. The model totals 545,098 parameters, matching the NumPy baseline.

5.5.3 Modular PyTorch Implementation

The network is implemented as an `nn.Module` subclass (`Cifar10CNN`), mirroring Fashion-MNIST except for the first convolution's channel count (3 instead of 1) and the flattened feature dimension ($64 \times 8 \times 8 = 4096$ instead of $64 \times 7 \times 7 = 3136$, because CIFAR-10's 32×32 inputs retain an 8×8 spatial dimension after two stride-2 poolings). The explicit training loop and `nn.CrossEntropyLoss()` carry over directly. Total parameters equal 545,098, matching the other two implementations.

5.6 Structural and Operational Framework Mapping

Table 7: CIFAR-10: component-by-implementation table

Component	Scratch	Keras	PyTorch
Data loading	Keras loader, shared NumPy arrays	same	same
Preprocessing	manual /255, channel-first reshape	/255, channel-last (as shipped, no transpose)	/255, channel-first (transposed)
Model definition	explicit functions + a params dict	`Sequential([...])`	`nn.Module` subclass
Convolution	`conv2d_forward/backward` (`im2col`, 3-channel-aware)	`Conv2D(...)`	`nn.Conv2d(...)`
Activation	`relu()`	`activation="relu"`	`nn.ReLU()`
Pooling	`max_pool2d_forward/backward`	`MaxPooling2D(2)`	`nn.MaxPool2d(2)`
Flatten	`flatten_forward/backward` (reshape)	`Flatten()`	`nn.Flatten()`
Dense/Linear layer	`linear_forward/backward`	`Dense(...)`	`nn.Linear(...)`
Loss	manual softmax cross-entropy	`SparseCategorical Crossentropy( from_logits=True)`	`nn.CrossEntropyLoss()`
Gradient	manual backprop	autodiff (implicit in `fit`)	`loss.backward()` (autograd)
Optimizer	manual `W -= lr * dW`	`optimizer="sgd"`	`torch.optim.SGD(...)`
Training loop	explicit `for epoch / for batch`	`model.fit(...)`	explicit `for epoch / for batch`
Evaluation	manual metric functions	`model.evaluate`-equivalent (manual)	manual (`model.eval()`, no-grad)
Prediction	`forward(X_test, params)`	`model.predict(...)`	`model(x_test)` under `torch.no_grad()`

5.7 Empirical Optimization Dynamics and Evaluation

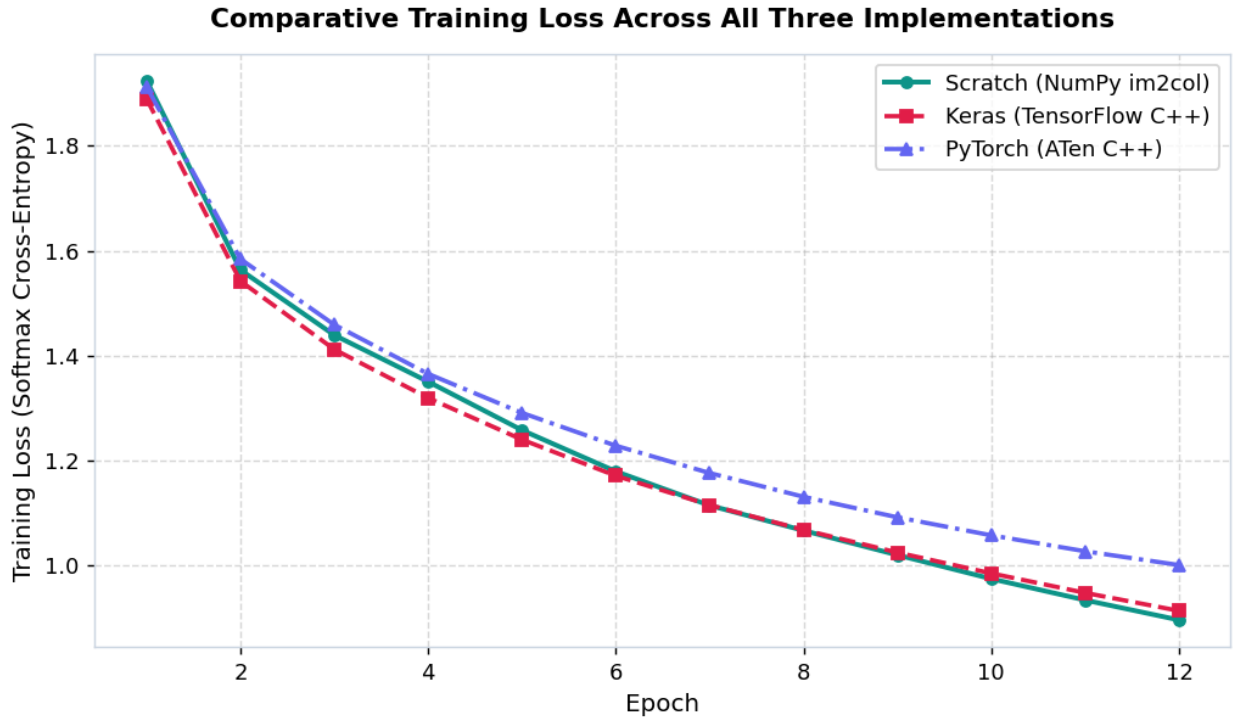


Figure 21: Comparative training loss trajectories across the three implementations on CIFAR-10 over 12 training epochs.

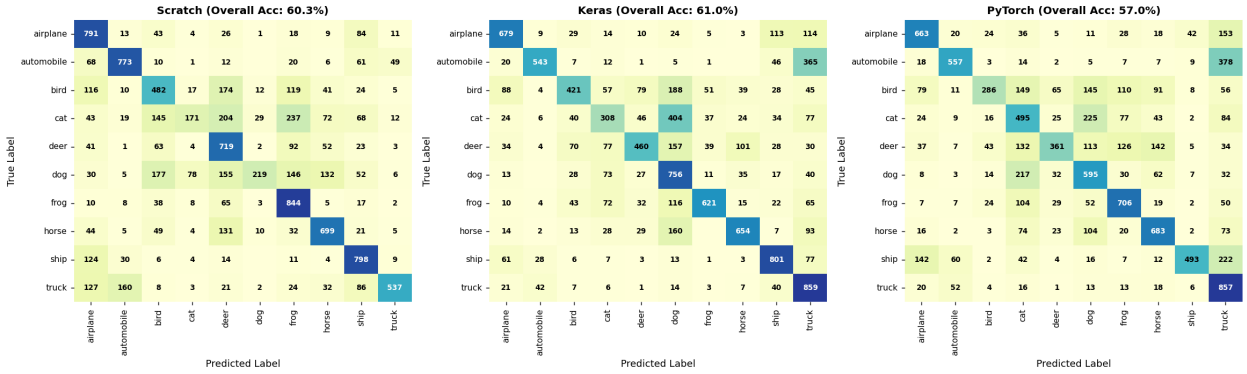


Figure 22: Normalized confusion matrices on the CIFAR-10 test set ($N = 10,000$), displaying consistent error clustering between semantically and visually adjacent pairs (cat/dog and automobile/truck).

Table 8: CIFAR-10: quantitative comparison, test set

Implementation	Params	Train time (s)	Final loss	Accuracy	Precision (macro)	Recall (macro)
Scratch	545,098	1,875.8	0.8965	0.6033	0.6335	0.6033
Keras	545,098	103.3	0.9854	0.6102	0.6477	0.6102
PyTorch	545,098	244.1	1.1220	0.5696	0.6163	0.5696

Macro F1 scores for the three implementations in the same order are: 0.5794 (Scratch), 0.6078 (Keras), and 0.5655 (PyTorch).

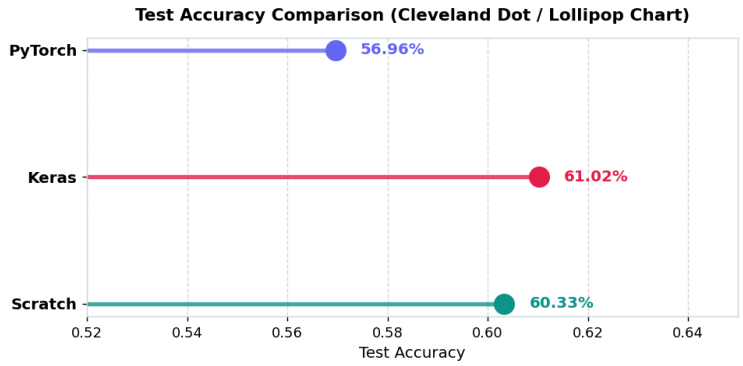


Figure 23: Cleveland dot / horizontal lollipop plot comparing CIFAR-10 test accuracy across implementations.

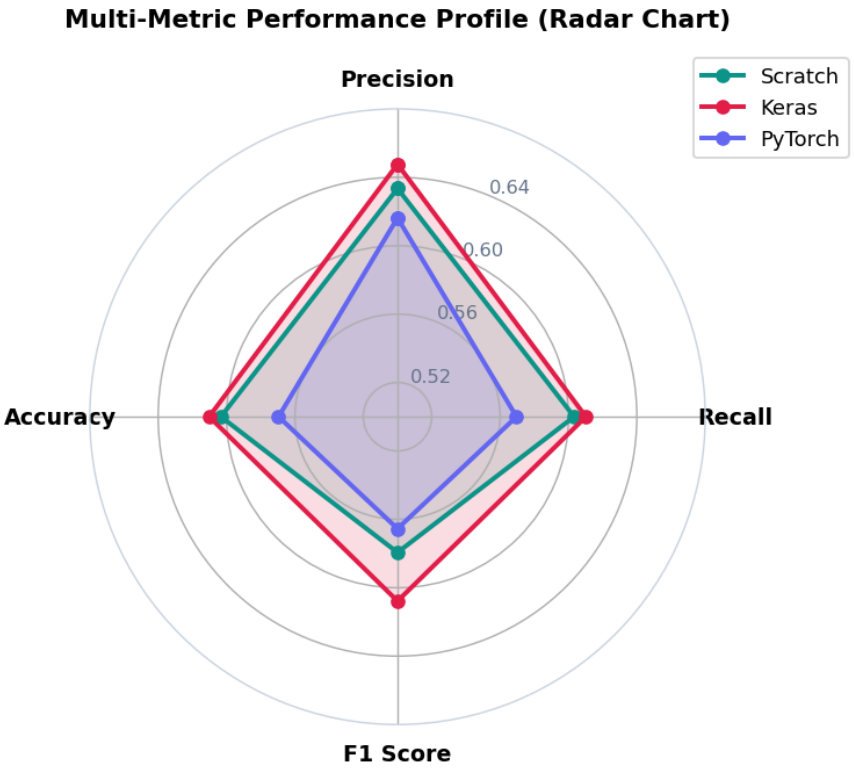


Figure 24: Multi-metric radar chart depicting macro-averaged Precision, Recall, F1 score, and Accuracy across all three implementations.

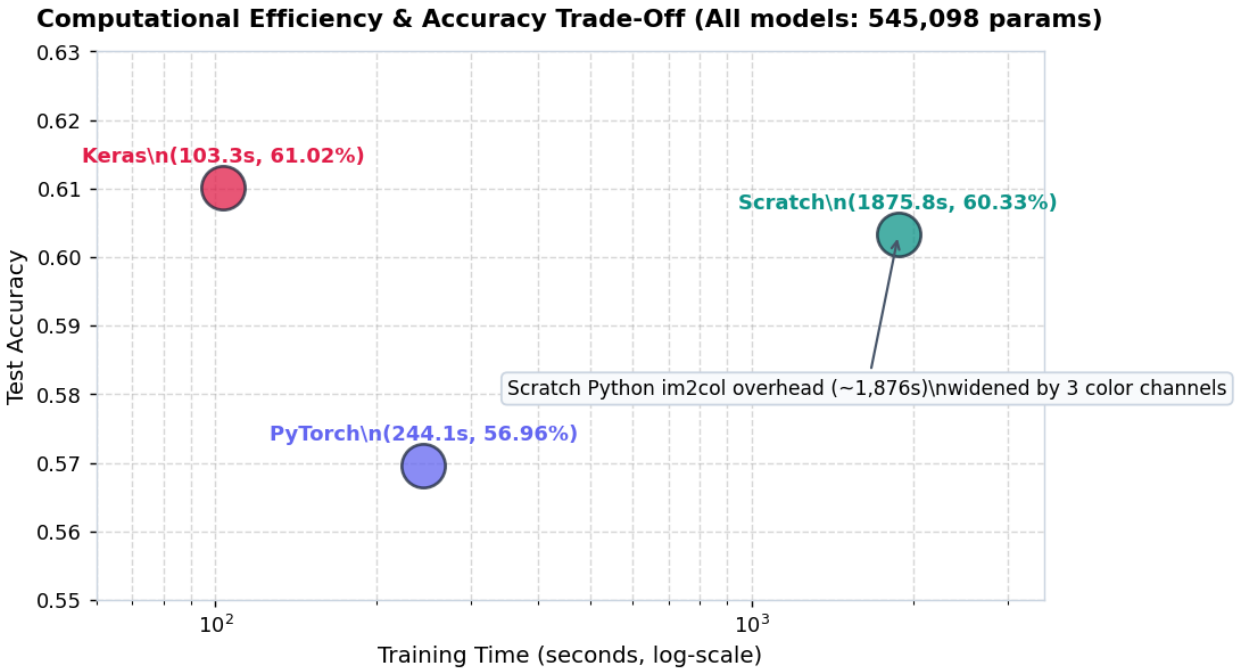


Figure 25: 2D Pareto efficiency trade-off chart illustrating the widened runtime gap on multi-channel convolutions between NumPy and compiled deep learning engines.

5.8 Comparative Analysis and Workload-Driven Latency

Keras finishes with a slight edge in both test accuracy (0.610) and F1 score (0.608), followed closely by Scratch (0.603 accuracy, 0.579 F1) and PyTorch (0.570 accuracy, 0.566 F1). All three implementations fall within approximately four percentage points of one another, maintaining the narrow performance spread observed in both preceding applications. The lower absolute accuracy (low 60s vs. high 80s for Fashion-MNIST) is expected: classifying photographic 32×32 color scenes with fine-grained class boundaries (cat vs. dog, automobile vs. truck) represents a much harder problem. A two-convolution network trained without data augmentation for only 12 epochs using vanilla SGD represents a modest baseline. The low 60s performance confirms the expected capacity of this architecture on this dataset, rather than any implementation deficiency.

Training duration repeats the pattern established in Fashion-MNIST, with Scratch exhibiting substantial execution latency that expands further here. Scratch required 1,875.8 seconds (approximately 31 minutes), Keras took 103.3 seconds, and PyTorch required 244.1 seconds. Scratch was approximately 18.2 times slower than Keras and 7.7 times slower than PyTorch, a wider multiple than Fashion-MNIST's 17.6x/6.9x, despite only a modest parameter increase (545,098 vs. 421,642). This highlights that spatial input dimensions and channel count ($32 \times 32 \times 3$ vs. $28 \times 28 \times 1$) drive the computational workload of uncompiled `im2col` convolutions, rather than parameter count alone. Scratch pays this cost directly in wall-clock time, while Keras and PyTorch leverage compiled multi-threaded routines that scale smoothly. Across the three applications, a clear progression emerges: on the diabetes MLP, Scratch is competitive and PyTorch is slowest; on the single-channel CNN, Scratch is 17.6x slower than Keras; on the 3-channel RGB CNN, Scratch is 18.2x slower than Keras. Handcrafted training cost scales with the volume of spatial convolution operations, not parameter count alone.

Abstraction trade-offs mirror those of Fashion-MNIST: Keras hid convolutions, pooling, differentiation, and batch scheduling behind declarative calls; PyTorch abstracted gradient computation; Scratch hid nothing. The cross-application pattern is established: accuracy spreads remain minimal whenever network architectures are rigorously controlled, validating Lecture 04's central hypothesis. Meanwhile, execution duration varies substantially based on hardware operation types, negligible on small dense networks, but pronounced and tied directly to convolution volume on CNNs.

6 Cross-Application Synthesis and Empirical Trends

Table 9 consolidates all three applications by test accuracy, the primary benchmark metric used across this report, across all three implementation paradigms. Every entry corresponds to values reported in Sections 3, 4, and 5.

Table 9: Cross-application summary: test accuracy, all 3 applications x 3 implementations

Application	Implementation	Parameters	Accuracy
Diabetes	Scratch	2,177	0.7310
	Keras	2,177	0.7533
	PyTorch	2,177	**0.7819**
Fashion-MNIST	Scratch	421,642	**0.8973**
	Keras	421,642	0.8603
	PyTorch	421,642	0.8660
CIFAR-10	Scratch	545,098	0.6033
	Keras	545,098	**0.6102**
	PyTorch	545,098	0.5696

Bold marks the winning implementation in each application. Parameter counts are identical across all three implementations within an application, confirmed by a direct assertion in every notebook.

Table 10: Cross-application training time, seconds, all 3 applications x 3 implementations

Application	Scratch	Keras	PyTorch
Diabetes	69.4	**24.7**	311.0
Fashion-MNIST	1,330.5	**75.6**	191.5
CIFAR-10	1,875.8	**103.3**	244.1

Bold marks the fastest implementation in each application. Keras is fastest in every application. Scratch is fastest of the two non-Keras implementations only on diabetes, its tiny MLP; scratch is the slowest implementation by a wide margin on both CNN applications.

No single framework dominates across all tasks, and the framework achieving the widest margin on diabetes (PyTorch) is simultaneously the slowest in that domain, demonstrating that accuracy and execution speed represent independent axes of software design.

7 Synthesis Discussion: Lessons Across Abstraction Levels

7.1 Validation of the Architectural Invariance Principle

Yes, on the axis it makes the strongest claim about. Every application's parameter count matched exactly across all three implementations, verified by direct programmatic assertions in each notebook environment. Furthermore, the accuracy spread across implementations remained bounded within four to five percentage points across all applications, the level of variation expected from pseudorandom initialization and batch

shuffling noise rather than divergent inductive capacity. Table 9's rotating winner (PyTorch on diabetes, Scratch on Fashion-MNIST, Keras on CIFAR-10) directly supports this hypothesis: if a particular library possessed an inherent modeling advantage, it would lead consistently rather than taking turns.

7.2 Divergence in Computational Latency and Kernel Scaling

Training time is where the three abstraction levels ceased to behave as interchangeable expressions of a computation. On the diabetes MLP, a compact fully-connected model, Scratch and Keras trained in comparable durations, and PyTorch was slowest due to per-batch autograd graph overhead on a 2,177-parameter model. Once spatial convolutions were introduced in Fashion-MNIST and CIFAR-10, this relationship inverted completely: Scratch became the slowest implementation by a wide margin (17.6x Keras on Fashion-MNIST, 18.2x on CIFAR-10). Handcrafted `im2col` convolutions dispatched through interpreted Python cannot compete with optimized, multi-threaded C++/CUDA kernels. The runtime gap widened between the two CNNs in step with image resolution and channel depth ($32 \times 32 \times 3$ vs. $28 \times 28 \times 1$) rather than parameter count. Full visibility and mathematical control incur tangible wall-clock execution costs.

7.3 Hidden Mechanics Surfaced Through Manual Implementation

Two key architectural insights emerged specifically from implementing training pipelines from scratch rather than relying on automated routines. First, in the diabetes task, PyTorch's loss converged to 0.8706 while Scratch and Keras converged near 0.51, not due to optimization error, but because `BCEWithLogitsLoss(pos_weight=...)` reweights only the positive class term while Keras's `class_weight` and Scratch reweight every observation. Surface-level naming similarities can mask meaningful underlying mechanical differences. Second, in Fashion-MNIST, the numerical gradient check detected a subtle bug during development: a redundant $1/N$ division in the dense backward pass that had been attenuating dense gradients. Because the network still trained with decreasing loss, this defect was invisible from external loss curves and was exposed solely by finite-difference gradient verification. Both findings demonstrate the profound diagnostic and pedagogical value of building neural components from scratch.

8 Conclusion and Key Takeaways

This report constructed three intelligent systems, diabetes risk screening (MLP), Fashion-MNIST apparel recognition (CNN), and CIFAR-10 object recognition (CNN), with each architecture implemented across three software paradigms: vectorized NumPy from scratch, TensorFlow/Keras, and PyTorch. Parameter counts matched bit-for-bit across implementations, and accuracy spreads remained within a few percentage points, confirming Lecture 04's core thesis: network architecture is invariant across abstraction levels, only its programmatic expression changes. Training duration diverged significantly on convolutional workloads: handcrafted NumPy execution cannot match mature framework optimization for spatial convolutions, with runtime scaling in proportion to input resolution and channel depth. Furthermore, writing training pipelines by hand surfaced critical implementation mechanics: the asymmetric scaling of PyTorch's `pos_weight` compared to symmetric weighting, and the discovery of a gradient scaling bug via numerical gradient verification. These findings illustrate the concrete trade-offs between convenience, visibility, and computational efficiency across software abstraction tiers.

9 System Environment and Reproducibility Protocol

9.1 Software Environment and Deterministic Controls

All three notebooks were built and executed in a dedicated virtual environment at `assignment_04/.venv` (Python 3.10.11, registered as Jupyter kernel `assignment_04`), with the following key package versions: numpy, pandas, scikit-learn (metrics only), matplotlib, seaborn, jupyter, nbconvert, nbformat, nbclient, tensorflow 2.21.0, torch 2.14.0+cpu, and torchvision 0.29.0+cpu. CPU-only PyTorch and torchvision distributions were installed deliberately to ensure execution time comparisons were not confounded by heterogeneous hardware acceleration. Complete deterministic reproducibility is maintained across all notebooks by fixing `RANDOM_SEED = 42` across `np.random.seed`, `random.seed`, `tf.random.set_seed`, and `torch.manual_seed`.

9.2 Headless Reproduction and Artifact Verification

Each application notebook is self-contained and can be executed headlessly from its own directory with:

```
../.venv/Scripts/python.exe -m jupyter nbconvert \
    --to notebook --execute --inplace \
    notebook/<app>.ipynb --ExecutePreprocessor.timeout=1800
```

replacing `<app>` with `diabetes`, `fashion_mnist`, or `cifar10`. Fashion-MNIST and CIFAR-10 retrieve their datasets automatically via `keras.datasets` on first execution. For the diabetes application, the CSV files must reside in `diabetes/data/`. Each notebook concludes by serializing its trained artifacts to its local `model/` directory:

- `scratch_weights.npz`: Trained parameters from the from-scratch implementation.
- `keras_model.keras`: Serialized Keras model archive.
- `pytorch_model.pt`: Serialized PyTorch state dictionary.
- `feature_names.joblib` (diabetes only): Post-encoding feature column order.
- `input_schema.json`: Formally documented input fields and prediction target.

followed by an automated verification cell that reloads all three artifacts and confirms their predictions match in-memory results computed during the same execution.